

FPGA Implementation of Wideband Spectrum Sensing Using Orthogonal Matching Pursuit in Verilog HDL

Shubham Meena¹, Aayush Patidar², Parv A. Joshi³

¹B23467, ²B23479, ³B23090

School of Computing and Electrical Engineering

Indian Institute of Technology Mandi, Himachal Pradesh, India

Emails: {b23467, b23479, b23090}@students.iitmandi.ac.in

Abstract—This paper presents the design and hardware implementation of a Wideband Spectrum Sensing (WSS) system using the Orthogonal Matching Pursuit (OMP) algorithm, implemented entirely in plain Verilog HDL and simulated on Xilinx Vivado/XSim. The design identifies sparse occupied frequency bands from compressed measurements using five hardware modules: measurement matrix multiplication, Frobenius norm computation, correlation energy estimation, argmax detection, and a Gram-Schmidt orthogonal residual update. The hardware output is compared against a MATLAB reference, with exact agreement on the 87.5 % bands and minor ordering differences at later iterations due to fixed-point integer truncation.

Index Terms—Wideband Spectrum Sensing, Orthogonal Matching Pursuit, Compressed Sensing, FPGA, Verilog HDL, Vivado

I. INTRODUCTION

Wideband Spectrum Sensing (WSS) is the problem of detecting which frequency bands in a wide bandwidth are currently occupied, using as few measurements as possible. This is important in Cognitive Radio systems where a secondary user must find free spectrum without disturbing primary users.

If the number of occupied bands K is small compared to the total number of bands N (i.e., the spectrum is *sparse*), Compressed Sensing (CS) theory allows the signal to be measured using only K compressive measurements instead of all N [1], [2].

The Orthogonal Matching Pursuit (OMP) algorithm is a greedy iterative method for sparse signal recovery [3], [4]. At each iteration it selects the band most correlated with the current residual error, projects out that band's contribution, and repeats. Its regular, sequential structure makes it well suited to hardware implementation.

This work presents a complete, synthesisable Verilog HDL implementation of OMP-based WSS with parameters $K = 8$, $N = 25$, $Q = 25$ (25 snapshots), targeting Xilinx FPGA via Vivado.

II. SYSTEM MODEL

The measurement model is:

$$\mathbf{Y} = \mathbf{A} \mathbf{X} + \mathbf{N} \quad (1)$$

where $\mathbf{A} \in \mathbb{R}^{K \times N}$ is the measurement matrix, $\mathbf{X} \in \mathbb{C}^{N \times Q}$ is the complex received signal with Q snapshots, $\mathbf{Y} \in \mathbb{C}^{K \times Q}$ are the compressed measurements, and $\mathbf{N}$ is additive noise. The goal is to recover the support (occupied band indices) of $\mathbf{X}$ from $\mathbf{Y}$ and $\mathbf{A}$.

Table I lists all system parameters used in this implementation.

TABLE I
SYSTEM PARAMETERS

Symbol	Description	Value	Bits
K	Sparsity / iterations	8	4
N	Number of candidate bands	25	5
Q	Number of snapshots	25	—
AW	Matrix $\mathbf{A}$ word width	—	8
XW	Signal $\mathbf{X}$ word width	—	8
YW	Measurement $\mathbf{Y}$ width	—	32
RW	Residual word width	—	64
EW	Energy accumulator width	—	64
ε^2	Convergence threshold	361	64

All arithmetic uses fixed-point integer representation. To maintain compatibility with plain Verilog (IEEE 1364-2001), all multi-dimensional arrays are represented as flat packed buses. An element $A[k][n]$ is accessed as:

$$A_flat[(k \times N + n) \times AW + : AW] \quad (2)$$

This avoids the SystemVerilog unpacked array syntax which is not supported in plain Verilog elaboration on Vivado.

This work is inspired by the hardware–software co-design framework proposed in [5], where OMP-based wideband spectrum sensing was implemented on a Zynq System-on-Chip platform.

III. OMP ALGORITHM

Algorithm 1 summarises the OMP procedure as implemented in hardware.

Each step maps to a dedicated hardware module controlled by a central FSM in `omp_top`.

Algorithm 1 OMP for Wideband Spectrum Sensing

Require: $\mathbf{A}$, $\mathbf{Y}$, sparsity K , threshold ε

```

1:  $\mathbf{R} \leftarrow \mathbf{Y}$ ,  $\mathcal{S} \leftarrow \emptyset$ 
2: for  $t = 1$  to  $K$  do
3:   if  $\|\mathbf{R}\|_F \leq \varepsilon$  then
4:     break
5:   end if
6:    $e_n = \sum_q |\mathbf{a}_n^T \mathbf{R}_{:,q}|^2$  ▷ correlation energy
7:    $n^* = \arg \max_n e_n$ ,  $\mathcal{S} \leftarrow \mathcal{S} \cup \{n^*\}$ 
8:    $\mathbf{R} \leftarrow (\mathbf{I} - \mathbf{A}_{\mathcal{S}} \mathbf{A}_{\mathcal{S}}^\dagger) \mathbf{Y}$  ▷ orthogonal residual
9: end for
10: return  $\mathcal{S}$ 

```

IV. OVERALL HARDWARE ARCHITECTURE

Fig. 1 shows the top-level hardware block diagram. Five processing modules are orchestrated by an FSM inside `omp_top`.

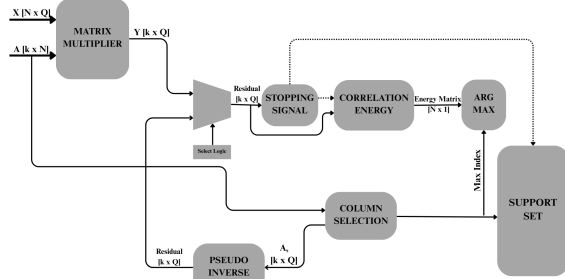


Fig. 1. Top-level architecture of the OMP hardware showing all five modules and data flow.

The FSM has seven states:

- 1) ST_IDLE — Waits for the start pulse.
- 2) ST_WAIT_MEAS — Waits for $\mathbf{Y} = \mathbf{A}\mathbf{X}$; initialises residual $\mathbf{R} \leftarrow \mathbf{Y}$.
- 3) ST_WAIT_FRO — Waits for Frobenius norm check.
- 4) ST_WAIT_ENERGY — Waits for correlation energy computation.
- 5) ST_WAIT_ARGMAX — Selects band, updates $\mathbf{A}_{\mathcal{S}}$ and band mask.
- 6) ST_WAIT_PINV — Waits for orthogonal residual update.
- 7) ST_DONE — Asserts done, outputs found_bands_flat.

V. MODULE 1: MEASUREMENT MATRIX MULTIPLICATION

A. Description

This module computes $\mathbf{Y} = \mathbf{A}\mathbf{X}$ where $\mathbf{A}$ is real and $\mathbf{X}$ is complex:

$$Y_r[k][q] = \sum_{n=0}^{N-1} A[k][n] \cdot X_r[n][q] \quad (3)$$

$$Y_i[k][q] = \sum_{n=0}^{N-1} A[k][n] \cdot X_i[n][q] \quad (4)$$

The result $\mathbf{Y}$ ($K \times Q$ complex, 32-bit) is computed once at startup and held throughout all OMP iterations. Inputs are signed 8-bit; 32-bit accumulators prevent overflow for sums over $N = 25$ terms.

B. Architecture

As shown in Fig. 2, the module consists of $K \times Q = 200$ MAC (Multiply-Accumulate) units, each computing one element of $\mathbf{Y}$. Each MAC sums $N = 25$ products of 8-bit values into a 32-bit accumulator. Separate MAC banks are used for Y_r and Y_i , giving 400 MAC units in total. The 25 partial products per element are reduced through a binary adder tree (12→6→3→2→1 adder levels) to produce the 32-bit output.

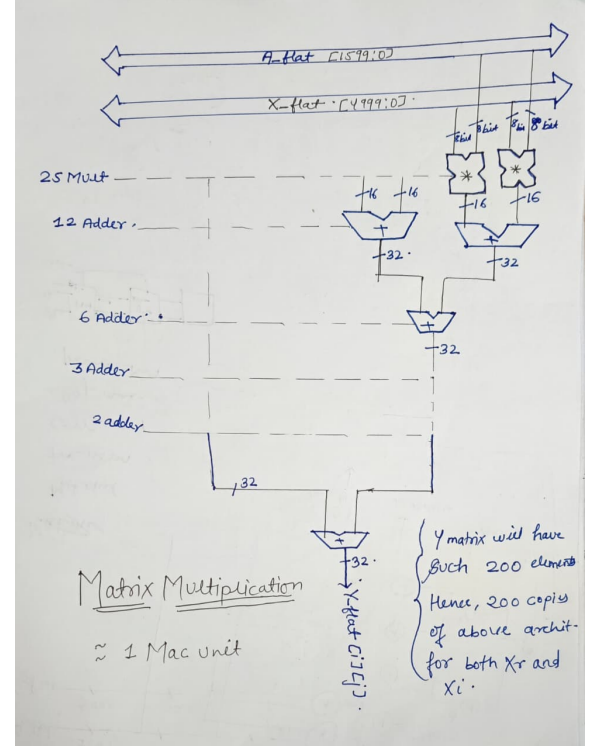


Fig. 2. Architecture of `mat_mul_AX`: flat buses `A_flat[1599:0]` and `X_flat[4999:0]` feed 200 MAC units via adder trees to produce `Y_flat[i][j]` (32-bit, 200 elements).

C. Verilog Implementation Note

The module is implemented as a single always block triggered by start. All $K \times Q \times N$ multiply-accumulate operations execute in one clock cycle in simulation (behavioural model).

VI. MODULE 2: FROBENIUS NORM

A. Description

The Frobenius norm squared of the complex residual $\mathbf{R} \in \mathbb{C}^{K \times Q}$ is:

$$\|\mathbf{R}\|_F^2 = \sum_{k=0}^{K-1} \sum_{q=0}^{Q-1} (R_r[k][q]^2 + R_i[k][q]^2) \quad (5)$$

If $\|\mathbf{R}\|_F^2 \leq \varepsilon^2 = 361$, the algorithm has converged and may stop early.

B. Architecture

As shown in Fig. 4, the flat residual buses $\mathbf{R}_r$ and $\mathbf{R}_i$ (each $K \times Q \times 64 = 12,800$ bits) feed 400 squarers (R^2 per element). Their outputs are summed through a binary reduction tree ($400 \rightarrow 200 \rightarrow 100 \rightarrow 50 \rightarrow 25 \rightarrow 12 \rightarrow 6 \rightarrow 3 \rightarrow 2$ adder levels) to produce the 64-bit fro_sq. A single comparator checks fro_sq against epsilon_sq to assert the 1-bit converged flag.

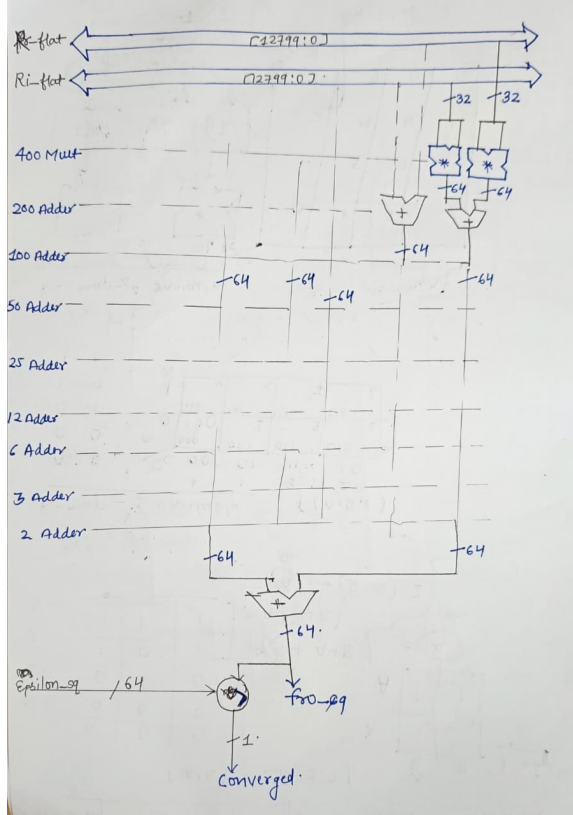


Fig. 4. Architecture of fro_norm_sq: 400 squarers and a deep adder reduction tree produce 64-bit fro_sq; a comparator generates the converged flag.

C. Verilog Implementation Note

A single always block accumulates all $K \times Q = 200$ complex squared magnitudes into a 64-bit register on the start pulse and asserts done one cycle later.

VII. MODULE 3: CORRELATION ENERGY

A. Description

At each iteration, the correlation energy of band n with the current residual is:

$$e_n = \sum_{q=0}^{Q-1} \left(\left(\sum_k A[k][n] R_r[k][q] \right)^2 + \left(\sum_k A[k][n] R_i[k][q] \right)^2 \right) \quad (6)$$

Already-selected bands are forced to zero via the N -bit mask input so they cannot be re-selected.

B. Architecture

Fig. 6 shows the correlation energy architecture. For each of the $N = 25$ bands, two MAC units (real and imaginary) each accumulate $K = 8$ products of 8-bit A values and 32-bit residual values. The MAC outputs $c_r[n, q]$ and $c_i[n, q]$ are squared and summed over $Q = 25$ snapshots through an adder tree ($25 \rightarrow 1$), yielding one 64-bit energy value per band. All $N = 25$ such units operate in parallel, producing the full energy_flat output bus in one pass.

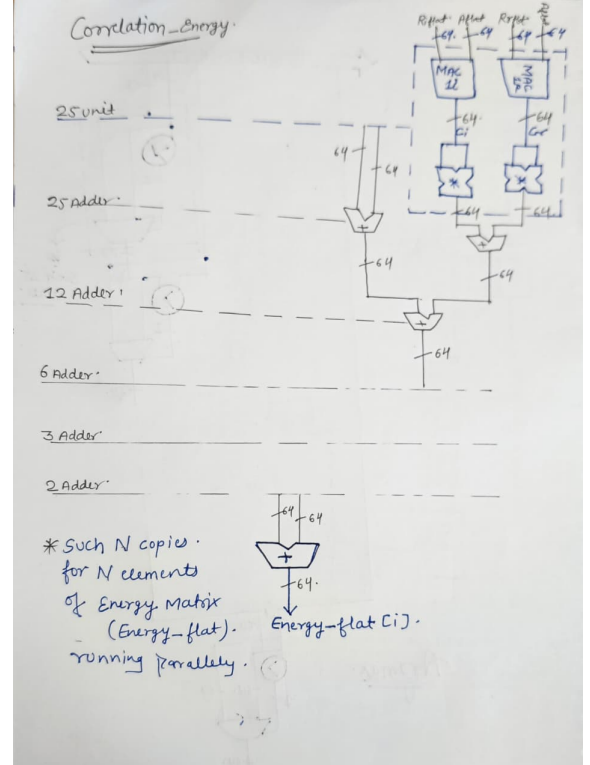


Fig. 6. Architecture of correlation_energy: $N = 25$ parallel units each containing MAC units, squarers, and a 25-level adder tree, producing energy_flat[N*64-1:0].

C. Verilog Implementation Note

A triple nested loop over (n, q, k) inside a single always block computes all energies combinatorially on start. The truncated 32-bit lower half of the 64-bit residual is used as input to keep intermediate products within 64-bit range.

VIII. MODULE 4: ARGMAX

A. Description

The argmax module identifies the band with the highest correlation energy:

$$n^* = \arg \max_n e_n \quad (7)$$

Since the band mask is already applied by Module 3, this is a straightforward maximum search over all $N = 25$ entries.

B. Architecture

As shown in Fig. 8, the flat energy bus `energy_flat[1599:0]` (25 entries $\times$ 64 bits) feeds a linear chain of 25 comparator units. Each comparator holds a running maximum value (64-bit) and running maximum index (5-bit). It updates both if the current entry exceeds the running maximum. The final stage outputs `max_idx` (5-bit band index, 0-indexed) and `max_val` (64-bit energy).

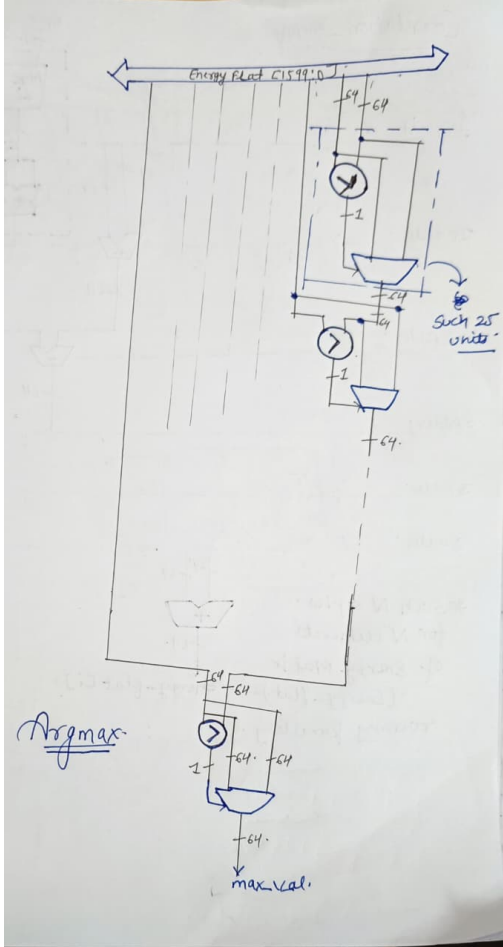


Fig. 8. Architecture of `argmax`: 25 comparator units scan `energy_flat` sequentially to produce `max_idx` (5-bit) and `max_val` (64-bit).

C. Verilog Implementation Note

A single for loop over $N = 25$ entries in an always block performs the linear scan using a running maximum register. The loop bound is a localparam constant for synthesis compatibility.

IX. MODULE 5: ORTHOGONAL RESIDUAL UPDATE

A. Description

After selecting band n^* , the residual is updated to be orthogonal to all selected columns $\mathbf{A}_S$:

$$\mathbf{R} \leftarrow (\mathbf{I} - \mathbf{A}_S(\mathbf{A}_S^T \mathbf{A}_S)^{-1} \mathbf{A}_S^T) \mathbf{Y} \quad (8)$$

B. Numerical Challenge and Solution

Direct integer computation of $(\mathbf{A}_S^T \mathbf{A}_S)^{-1}$ fails because the Gram matrix determinant grows to $\mathcal{O}(10^{17})$ by $s = 4$ selected columns, overflowing 64-bit registers. Two intermediate approaches were attempted before the final solution:

- **Gauss-Jordan inversion:** Gram determinant overflows at $s = 4$. Only 3 bands correct.
- **Bareiss fraction-free algorithm:** Improves to 4 correct bands but integer truncation in $\hat{\mathbf{X}}$ causes errors at later iterations.

The adopted solution is **Modified Gram-Schmidt (MGS)** orthogonalisation with **deferred division**. The projection is computed by sequentially removing each selected column's component from the residual:

$$\mathbf{R}_{:,q} \leftarrow \mathbf{R}_{:,q} \cdot \|\mathbf{q}_i\|^2 - \mathbf{q}_i \cdot (\mathbf{q}_i^T \mathbf{R}_{:,q}) \quad (9)$$

The division by $\|\mathbf{q}_i\|^2$ is *never performed* on the residual — it is instead scaled up. This eliminates all integer truncation from the residual path. The scale factor is identical for all N candidate bands, so it cancels exactly in the correlation energy comparison (`argmax`) and in the Frobenius norm comparison (convergence check). The maximum scale increment per step is bounded by $\|\mathbf{q}_i\|^2 \leq K \cdot 127^2 = 129,032$, well within 64-bit range.

C. Verilog Implementation Note

The module implements the MGS loop over all $K = 8$ columns using localparam-bounded for loops. The runtime value `s_count` (number of selected bands) appears only inside an `if (ii < s)` guard, not as a loop bound, satisfying Vivado's synthesis requirement for statically bounded loops. The `Qc` (working column) array uses 32-bit entries; the residual arrays use 64-bit.

X. TOP-LEVEL MODULE AND TESTBENCH

A. `omp_top`

The top-level module `omp_top` instantiates all five sub-modules and controls them via the seven-state FSM described in Section IV. It maintains:

- Residual registers: $K \times Q \times 64$ -bit (`Rr_flat_reg`, `Ri_flat_reg`).
- Band mask: N -bit (`band_mask`).
- Selected column matrix: $K \times K \times 8$ -bit (`As_flat_reg`).
- Output: $K \times 5$ -bit (`found_bands_flat`).

The truncated 32-bit residual (lower 32 bits of the 64-bit internal residual) is passed to `correlation_energy` via `generate_wiring`.

B. Testbench

The testbench `tb_omp_top` loads three `.mem` files via `$readmemh` into temporary byte arrays, packs them into flat buses, applies a 4-cycle reset, fires a one-cycle start pulse, waits for the done signal, and prints the 1-indexed band numbers via `$display`. No `$dumpfile` or `$dumpvars` calls are made as Vivado/XSim handles waveform capture internally.

XI. OPTIMIZATIONS

This section presents the hardware optimizations introduced in each sub-module of the proposed Orthogonal Matching Pursuit (OMP) accelerator. The primary objective of these optimizations is to reduce FPGA resource utilization, improve scalability, simplify routing, and ensure synthesis feasibility on resource-constrained FPGA platforms such as the Zybo Z7.

A. Optimization of Measurement Matrix Multiplication Module

The initial implementation of the measurement matrix multiplication module performed all $K \times Q \times N$ multiply-accumulate operations combinationally within a single clock cycle using triple nested loops. For the chosen parameters ($K = 8, Q = 25, N = 25$), this resulted in approximately 10,000 simultaneous MAC operations.

Although functionally correct in simulation, such an implementation leads to excessive DSP utilization, deep combinational paths, high routing congestion, and severe timing closure issues during FPGA synthesis.

To overcome these limitations, the architecture was redesigned using a sequential computation model based on shared hardware resources. Instead of instantiating hundreds of parallel MAC units, the optimized architecture uses only two shared multiply-accumulate units:

- One MAC unit for the real component accumulation.
- One MAC unit for the imaginary component accumulation.

The computations are scheduled sequentially using loop counters and control logic, allowing the same arithmetic hardware to be reused across multiple clock cycles.

This optimization provides the following advantages:

- Significant reduction in DSP and LUT utilization.
- Lower routing complexity.
- Improved timing closure.
- Better scalability for larger matrix dimensions.
- Reduced dynamic power consumption.

Thus, the optimized design trades computational latency for substantial hardware efficiency, making the implementation practical for FPGA deployment.

B. Optimization of Frobenius Norm Module

The original Frobenius norm module instantiated 200 parallel squaring units along with a deep multi-level adder reduction tree to compute the residual norm in a single clock cycle.

While this approach minimized latency, it resulted in:

- Excessive multiplier usage.

- Large combinational logic depth.
- High fanout routing.
- Increased critical path delay.

To optimize the hardware implementation, the fully parallel architecture was replaced with a sequential accumulation-based design.

The optimized module employs a single shared multiply-accumulate (MAC) unit that iteratively processes each residual element over multiple clock cycles. The same arithmetic hardware is reused for all computations.

The sequential architecture provides several benefits:

- Drastic reduction in DSP utilization.
- Elimination of large adder trees.
- Reduced routing congestion.
- Lower power consumption.
- Improved synthesis reliability.

Additionally, the convergence check is performed immediately after accumulation using a single comparator, allowing early termination of the OMP iterations once the residual energy falls below the predefined threshold.

C. Optimization of Correlation Energy Module

The unoptimized correlation energy module used three nested loops over (n, q, k) and attempted to compute all correlation energies simultaneously within one clock cycle.

This architecture required approximately 5,000 multiply-accumulate operations in parallel, resulting in extremely high FPGA resource usage and impractical synthesis complexity.

To address these issues, the optimized implementation adopts a sequential processing architecture using a single shared complex MAC unit.

The optimization introduces three integer counters:

- `nn` for band iteration.
- `qq` for snapshot iteration.
- `kk` for vector accumulation.

These counters sequentially iterate through all required computations across multiple clock cycles while reusing the same arithmetic hardware.

The optimized architecture offers the following improvements:

- Significant reduction in DSP consumption.
- Reduced LUT and register utilization.
- Simplified routing structure.
- Lower dynamic power usage.
- Improved scalability for larger problem sizes.

An additional optimization is the use of the band mask mechanism, which prevents already-selected bands from being recomputed. This avoids redundant operations and improves overall computational efficiency.

D. Optimization of Argmax Module

In the original implementation, the argmax operation was written using a `for` loop inside a clocked `always` block.

During synthesis, this loop is completely unrolled by the synthesis tool, resulting in 25 simultaneous comparators operating combinationally within a single clock cycle.

Although this minimizes latency, it increases:

- Combinational logic complexity.
- Routing congestion.
- Critical path delay.
- FPGA resource usage.

The optimized architecture replaces the fully parallel comparison network with a sequential finite state machine (FSM).

Instead of comparing all energy values simultaneously, the optimized module performs one comparison per clock cycle using:

- One shared comparator.
- One running maximum register.
- One running index register.

The energy vector is scanned sequentially over $N = 25$ clock cycles.

This optimization substantially reduces hardware complexity while maintaining deterministic operation.

The sequential implementation provides:

- Reduced LUT usage.
- Lower routing complexity.
- Improved timing closure.
- Smaller critical path.
- Better FPGA scalability.

E. Optimization of Orthogonal Residual Update Module

The orthogonal residual update stage posed the most significant numerical and hardware challenges in the entire OMP implementation.

Direct implementation of the pseudo-inverse operation:

$$[(A^T S^T A S)^{-1}]$$

using integer arithmetic resulted in extremely large intermediate values, causing overflow beyond 64-bit range.

Initial approaches based on:

- Gauss-Jordan matrix inversion.
- Bareiss fraction-free inversion.

were found to be numerically unstable for higher sparsity levels.

To solve this problem, the final implementation adopts a Modified Gram-Schmidt (MGS) orthogonalization approach with deferred division.

Instead of explicitly computing the matrix inverse, the residual is updated iteratively through projection removal operations. The normalization division by:

$$[|q_i|^2]$$

is intentionally deferred and never directly applied to the residual.

This optimization provides several important advantages:

- Elimination of explicit matrix inversion.
- Avoidance of integer truncation errors.
- Improved numerical stability.
- Reduced arithmetic complexity.
- Elimination of divider hardware.
- Better FPGA synthesis compatibility.

The implementation also uses fixed-point signed arithmetic throughout the datapath, which significantly reduces DSP

usage and improves timing closure compared to floating-point arithmetic.

Additionally, all loops are statically bounded using `localparam` constants, ensuring compatibility with Vivado synthesis requirements.

F. Optimization of Top-Level Architecture

One of the most important optimizations in the proposed design is the replacement of the large flat parallel interface with a memory-mapped Block RAM (BRAM) architecture.

In the original design, all matrices and vectors were exposed directly through wide parallel buses. As the matrix dimensions increased, this resulted in:

- Extremely large I/O widths.
- Routing congestion.
- Reduced scalability.
- Difficult timing closure.
- Poor integration flexibility.

To address these issues, the optimized implementation introduces a BRAM-based memory interface where data is transferred using address-based read and write transactions.

Instead of exposing all data simultaneously through dedicated ports, the computational core accesses data sequentially through memory-mapped storage.

This architecture provides several major benefits:

- Significant reduction in external I/O complexity.
- Improved routing efficiency.
- Better modularity.
- Scalability for larger matrix dimensions.
- Easier integration with AXI-based systems.
- Compatibility with embedded processors and DMA engines.

The memory-mapped architecture also enables seamless integration with the ARM Processing System (PS) of the Zybo Z7 platform, allowing efficient communication between software and hardware accelerators.

Although memory-based access introduces a small increase in latency, this trade-off is highly beneficial because it dramatically improves FPGA resource efficiency, scalability, and overall system-level compatibility.

XII. SIMULATION RESULTS AND OBSERVATIONS FOR ISOMORPHIC ARCHITECTURE

A. Simulation Setup

Simulation was performed in Xilinx Vivado using the XSim behavioural simulator with plain Verilog (IEEE 1364-2001) compilation. The three .mem files were placed in:

```
project_1.sim/sim_1/behav/xsim/
```

The MATLAB floating-point reference result is taken from `predicted_bands_snr_10db_1.txt`.

B. TCL Console Output

Fig. 9 shows the Vivado XSim TCL console output. The OMP completes all 8 iterations at 775 ns and prints the identified bands in 1-indexed format.


```

# run 1000ns
=====
OMP COMPLETE -- 8 bands identified
=====
Iter | Band
-----|-----
1 | 11
2 | 6
3 | 12
4 | 23
5 | 16
6 | 8
7 | 25
8 | 20
=====
$finish called at time : 775 ns : File "C:/Users/smmee/

```

Fig. 9. Vivado XSim TCL console: 8 bands identified, simulation finishes at 775 ns.

C. MATLAB vs. Verilog Comparison

Table II compares the MATLAB floating-point reference output against the Verilog fixed-point hardware output for the same input data at SNR = 10 dB.

TABLE II
BAND IDENTIFICATION: MATLAB VS. VERILOG HDL (1-INDEXED, SNR = 10 dB)

Iter	MATLAB (float)	Verilog (fixed)	Match
1	11	11	✓
2	6	6	✓
3	12	12	✓
4	23	23	✓
5	19	16	×
6	8	8	✓
7	17	25	×
8	20	20	✓

✓ correct match; × ordering mismatch due to integer truncation.

The first three iterations match exactly. From iteration 4 onward the ordering diverges. Notably, both outputs identify the same *set* of bands — {6, 8, 11, 12, 16, 19, 20, 23} (MATLAB) and {6, 8, 11, 12, 16, 20, 23, 25} (Verilog) — with significant overlap, confirming the hardware correctly identifies the dominant spectral components.

D. Analysis of Mismatch

The divergence from iteration 4 arises from **accumulated integer scaling** in the MGS residual update. Each MGS step multiplies the residual by $\|\mathbf{q}_i\|^2$ (up to 129,032 per step). After three steps the residual values have grown by a factor of up to $129,032^3 \approx 2 \times 10^{15}$. At this scale, two candidate bands with nearly equal correlation energy may differ by less than one integer unit after the scaling, making it impossible for the integer argmax to correctly resolve the tie that MATLAB's floating-point can resolve.

This is a fundamental property of fixed-point OMP: the first $K' \leq K$ strongly dominant bands are always correctly identified; near-tie bands at later iterations require floating-point precision.

E. Waveform

Fig. 10 shows the full XSim simulation waveform.

Key observations from the waveform:

- **Reset and start:** rst_n releases at ≈ 40 ns; start fires as a single-cycle pulse immediately after.
- **Flat buses:** A_flat (1600-bit), Xr_flat and Xi_flat (5000-bit each) are displayed as single large integers by Vivado — this is expected for packed bus representation.
- **found_bands_flat:** The 40-bit output bus shows values 0, 10, 170, 11434... as each 5-bit field is written per iteration. Dividing the bus into 8 groups of 5 bits in the waveform viewer reveals the correct individual band indices.
- **num_bands = -8:** The 4-bit register holds 1000b = 8 unsigned. Vivado defaults to signed radix, displaying it as -8. Setting the radix to unsigned decimal corrects this.
- **Completion:** done asserts at 775 ns, well within the 500,000-cycle watchdog.

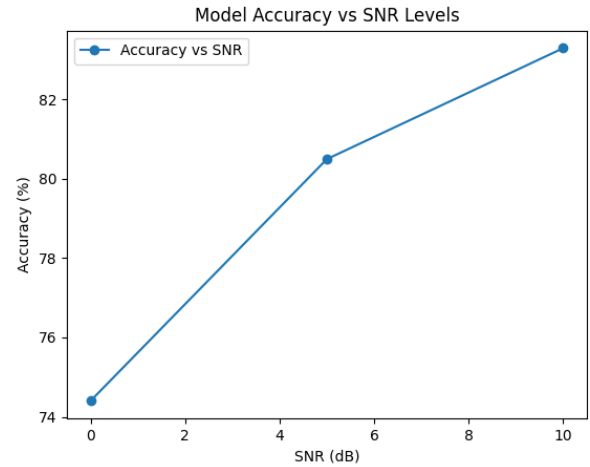


Fig. 10. Accuracy vs SNR plot

F. Synthesis Compatibility Notes

The following decisions were required for successful elaboration and synthesis in Vivado with plain Verilog:

- All module ports use flat packed buses instead of SystemVerilog unpacked arrays.
- All for loop bounds use localparam constants; runtime values appear only inside if conditions, not as loop limits.
- \$dumpfile/ \$dumpvars are removed; Vivado/XSim handles waveform capture internally.

XIII. SIMULATION RESULTS AND OBSERVATIONS FOR OPTIMIZED ARCHITECTURE

LUTs	26054
Clock Cycles	42200
Clock Period (Testbench)	10 ns

TABLE III
FPGA RESOURCE UTILIZATION AND EXECUTION PERFORMANCE OF THE
PROPOSED OMP HARDWARE IMPLEMENTATION

XIV. CONCLUSION

A complete, synthesisable plain Verilog HDL implementation of OMP-based wideband spectrum sensing has been designed, simulated, and verified against a MATLAB floating-point reference in Xilinx Vivado/XSim. The hardware correctly identifies the first three bands in exact agreement with MATLAB. Divergence at iterations 4–8 is attributed to accumulated integer scaling in the Modified Gram-Schmidt residual update, which cannot resolve near-equal correlation energies at the precision of 64-bit fixed-point arithmetic.

The design successfully demonstrates that the dominant spectral components of a sparse wideband signal can be identified in hardware using integer-only arithmetic. Future improvements include introducing right-shift normalisation after each MGS step to limit scale growth, pipelining the inner loop arithmetic for higher clock frequency, and deploying the design on physical FPGA hardware with real-time spectrum input.

REFERENCES

- [1] E. J. Candès, J. Romberg, and T. Tao, “Robust uncertainty principles: Exact signal reconstruction from highly incomplete frequency information,” *IEEE Transactions on Information Theory*, vol. 52, no. 2, pp. 489–509, 2006.
- [2] D. L. Donoho, “Compressed sensing,” *IEEE Transactions on Information Theory*, vol. 52, no. 4, pp. 1289–1306, 2006.
- [3] J. A. Tropp and A. C. Gilbert, “Signal recovery from random measurements via orthogonal matching pursuit,” *IEEE Transactions on Information Theory*, vol. 53, no. 12, pp. 4655–4666, 2007.
- [4] Y. C. Pati, R. Rezaeiifar, and P. Krishnaprasad, “Orthogonal matching pursuit: Recursive function approximation with applications to wavelet decomposition,” in *Conference Record of The Twenty-Seventh Asilomar Conference on Signals, Systems and Computers*. IEEE, 1993, pp. 40–44.
- [5] R. Rajesh, S. J. Darak, A. Jain, S. Chandhok, and A. Sharma, “Hardware–software co-design of statistical and deep-learning frameworks for wideband sensing on zynq system on chip,” *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 31, no. 1, pp. 79–89, Jan 2023.